

Guia de Estudo do Projeto Flutter (pasta lib)

Objetivo deste material: explicar de forma didática como seu projeto está estruturado hoje, o que cada arquivo faz, quais widgets existem, por que certas decisões foram tomadas e quais pontos ainda são placeholders para lógica de negócio.

Versão analisada: estado atual da pasta lib em 24/03/2026.

Importante: hoje existe um caminho principal de execução (ShellPage) e alguns

arquivos legados/referência (home_page.dart e route_transition.dart) que

ajudam no estudo, mas não são o fluxo central da UI atual.

1) Visão Geral da Arquitetura Atual

O app está organizado em três níveis principais:

- **Entrada da aplicação:** main.dart.
- **Configuração de app e rotas:** src/wireframe.dart.
- **Tela container principal:** src/pages/shell_page.dart, que mantém header/navbar fixos e troca apenas o conteúdo central.

Esse desenho é muito bom para UX porque:

- evita reconstruções desnecessárias de elementos estáveis (header e navbar),
- permite transições mais suaves entre conteúdos (Projetos e Ideias),
- simplifica o gerenciamento visual da aplicação.

2) Fluxo de Execução (do boot até a tela)

2.1 `lib/main.dart`

É o ponto de entrada padrão do Flutter.

- Função `main()` chama `runApp(const Wireframe())`.
- Isso monta o widget raiz do app.

2.2 `lib/src/wireframe.dart`

Define o `MaterialApp` e as rotas nomeadas.

- `initialRoute: "/projects"` define a aba inicial.
- Rotas `"/"`, `"/projects"` e `"/ideas"` todas criam `ShellPage`, variando o `initialTab`.

Decisão importante: hoje a navegação principal entre Projetos/Ideias acontece por estado interno do `ShellPage` (mais fluido), não por troca constante de rota.

3) Tela Principal Container

3.1 `lib/src/pages/shell_page.dart`

Este é o coração da UI atual.

Widget: `ShellPage` (`StatefulWidget`).

Estado:

- `_activeTab` (`NavTab.projects` ou `NavTab.ideas`): define o conteúdo ativo.
- `_toIdeas` (`bool`): guarda a direção da animação horizontal.

3.2 Estrutura visual do `Scaffold`

- `backgroundColor`: tom rosado base.
- `bottomNavigationBar`: `CustomNavBar` com callback `onTabSelected`.
- `floatingActionButton`: botão "+" (ainda sem lógica).
- `body`: `Stack` com:

- fundo global por imagem `assets/images/FUNDO.png`,
- camada principal em `Column` com `MainHeader`, `FuncoesBusca` e área de conteúdo animada.

3.3 Animação entre Projetos e Ideias

Feita por `_AnimatedTabContent` usando `AnimatedSwitcher`:

- **entrada:** slide horizontal + fade;
- **saída:** slide para o lado oposto + leve desvanecimento;
- **direção:** controlada por `_toIdeas` para manter semântica correta (Projetos → Ideias vs Ideias → Projetos).

Decisão técnica: usar `KeyedSubtree` com `ValueKey(activeTab)` para o Flutter entender que são conteúdos distintos e animar corretamente.

3.4 Conteúdo de Projetos

`_ProjectsContent` usa `ListView.builder` e cria 4 `ProjectCard` de exemplo.

3.5 Conteúdo de Ideias

Quando `activeTab == ideas`, o shell renderiza `IdeasContent` (arquivo `ideas_page.dart`).

4) Página de Ideias e Subseções

4.1 `lib/src/pages/ideas_page.dart`

Widget principal: `IdeasContent` (`StatefulWidget`).

- Estado interno `IdeasView _activeView` alterna entre `notes` e `diagrams`.
- Topo com dois botões reutilizáveis: “Notas” e “Diagramas”.
- Título de contexto (“Notas recentes” / “Grupos de diagramas”).
- Conteúdo trocado por `AnimatedSwitcher` com transição vertical curta + fade.

4.2 Subtela Notas (`_NotesSubPage`)

Exibe lista mock de pastas e ideias relacionadas à escrita criativa:

- `_FolderCard`: cards para agrupamento temático (ex.: universo, personagens).
- `_IdeaCard`: cards para notas individuais (ex.: conflito central, virada de ato).

4.3 Subtela Diagramas (`_DiagramsSubPage`)

Exibe grupos de diagramas de história com aparência mais editorial:

- `_DiagramGroupCard` com `title` e `subtitle`.
- “chips” internos (`_DiagramChip`) para metadados visuais.

4.4 Sistema de superfície visual

`_IdeasSurfaceCard` centraliza estilo visual de cards (fundo translúcido, borda clara, sombra suave), reduzindo repetição e mantendo consistência entre itens.

4.5 Interatividade atual nessa tela

- **Existe:** alternância entre “Notas” e “Diagramas”.
- **Ainda não existe:** abrir pasta, abrir nota, editar item, reorder, persistência.

5) Widgets Compartilhados da Interface

5.1 `lib/src/widgets/main_header.dart`

Widget do cabeçalho gradiente “WIREFRAME”.

- Parâmetro `asSliver` permite reutilizar de dois jeitos:
- `true`: como `SliverAppBar` (cenário de scroll com slivers).
- `false`: como bloco fixo em `SizedBox` (cenário do shell atual).

Decisão: boa reutilização sem duplicar layout de header.

5.2 `lib/src/widgets/funcoes_busca.dart`

Barra de ações de busca e campo de texto.

- Ícones: filtro, ordenação e “mágica” (todos ainda sem ação).
- `TextField` visual para busca (sem controller/lógica de filtragem por enquanto).

5.3 lib/src/widgets/custom_nav_bar.dart

Navbar inferior personalizada com duas abas.

- `enum NavTab { projects, ideas }`.
- Recebe `activeTab` para destaque visual.
- Recebe `onTabSelected` para comunicação com widget pai (shell).

Decisão: componente “burro” (stateless), deixando estado no container pai. Isso melhora testabilidade e previsibilidade.

5.4 lib/src/widgets/ideas_toggle_button.dart

Botão reutilizável da subtela Ideias (Notas/Diagramas).

- Parâmetros: `label`, `isActive`, `onTap`.
- Usa `AnimatedContainer` e `AnimatedDefaultTextStyle` para transição suave de estado ativo/inativo.

Decisão: evita duplicação e concentra identidade visual do toggle.

5.5 Botões do header

- `lib/src/widgets/buttons/botao_voltar.dart`
- `lib/src/widgets/buttons/botao_config.dart`

Ambos são botões circulares translúcidos com ícones. Atualmente, `onPressed` está vazio (placeholder).

5.6 lib/src/widgets/project_card.dart

Card de projeto com expansão/colapso animados (um dos widgets mais completos hoje).

Principais pontos técnicos:

- `StatefulWidget` com `AnimationController` próprio.
- Transição de expansão com `SizeTransition` + `FadeTransition`.
- Raio inferior do cabeçalho animado para evitar “snap” visual.
- Conteúdo detalhado extraído para `_ProjectDetails` (separação interna limpa).

Interação atual: só expandir/recolher; sem persistência real de dados.

6) Arquivos de Referência / Legado

6.1 `lib/src/pages/home_page.dart`

Representa uma versão anterior da estrutura (com `CustomScrollView` e `slivers`). Não é o fluxo principal hoje, mas é útil como referência de composição.

6.2 `lib/src/navigation/route_transition.dart`

Arquivo criado para estratégia anterior de transição por rota nomeada. No estado atual, o shell controla transição por estado interno, então este arquivo está ocioso.

Resumo prático: os dois arquivos acima podem ser mantidos para estudo histórico ou

removidos numa limpeza futura para reduzir ruído.

7) Decisões de Design e Engenharia (por que assim?)

- **Header/Navbar estáticos:** estabilidade visual e menor custo de reconstrução.
- **Animação apenas no conteúdo:** percepção de fluidez e foco no que realmente muda.
- **Componentização:** botões, navbar, cards e toggles separados para reaproveitamento.
- **Mocks de conteúdo:** acelera validação de UX antes da lógica de negócio.
- **Uso de `withValues(alpha: ...)`:** API atual recomendada para opacidade em cores.

8) Estado Atual da Lógica de Negócio

Hoje a lógica ainda é majoritariamente de UI. O que já existe:

- estado local de aba principal (Projetos/Ideias),
- estado local de subtela em Ideias (Notas/Diagramas),
- estado local de expansão em cada card de projeto.

O que ainda falta para negócio real:

- modelos de domínio (`Project`, `Idea`, `Character`, etc.),

- repositórios e persistência (local/remoto),
- casos de uso (criar, editar, excluir, filtrar, ordenar),
- conexão da busca/filtros com dados reais.

9) Como Estudar Este Projeto (roteiro recomendado)

1. Leia `main.dart` e `wireframe.dart` para entender a entrada e rotas.
2. Estude `shell_page.dart` para dominar estrutura e animações de troca de conteúdo.
3. Explore `ideas_page.dart` para aprender composição de subviews e reuso de estilo.
4. Analise `project_card.dart` para aprender animação custom com controller.
5. Volte aos widgets compartilhados para entender padrões de reutilização.

10) Glossário Rápido dos Conceitos Flutter usados

- **StatelessWidget:** widget sem estado mutável interno.
- **StatefulWidget:** widget com estado interno e `setState()`.
- **Scaffold:** estrutura base de tela Material (body, FAB, bottom bar).
- **AnimatedSwitcher:** troca entre filhos com animação automática.
- **AnimationController:** controle explícito de animações.
- **ListView.builder:** lista eficiente para itens repetidos.
- **InkWell:** resposta visual de toque para áreas clicáveis.

11) Conclusão

Seu projeto já está com uma base visual sólida e modular para evoluir. A principal conquista arquitetural foi centralizar a navegação visual no `ShellPage`, mantendo elementos estáticos e animando somente a área variável. Isso facilita muito a próxima fase: adicionar lógica de negócio sem precisar reescrever o design principal.

Documento gerado automaticamente para estudo do projeto atual.